

PROGRAMACIÓN WEB EN EL ENTORNO SERVIDOR

Introducción a APIs y Principios RESTful



Objetivos de Aprendizaje

- Comprender qué es una API y su importancia en el desarrollo web
- Conocer los diferentes tipos de APIs web (REST, GraphQL, SOAP)
- Entender los principios fundamentales de REST
- Identificar las características de una API RESTful bien diseñada



¿Qué es una API?

- **Definición simple:** una API es como un "camarero" entre dos sistemas
- **Función:** Permite que diferentes aplicaciones se comuniquen entre sí
- **Analogía del restaurante:**
 - Cliente (tu aplicación) -> Camarero (API) -> Cocina (servidor/base de datos)
 - El camarero lleva tu pedido y trae la comida sin que sepas cómo se cocina

El potencial primario de las APIs es que permiten la comunicación sin revelar la **implementación interna**.



APIs que Usamos Todos los Días

- **Redes sociales:** Compartir contenido entre Facebook, Instagram, etc.
- **Mapas:** Google Maps integrado en aplicaciones de reparto de comida
- **Pagos:** PayPal, Stripe en tiendas online
- **Clima:** Información meteorológica en aplicaciones móviles
- **Autenticación:** Iniciar sesión con Google/Facebook



Elementos Clave de una API Web

- **Endpoint:** La URL específica donde se hace la petición
 - ['https://api.ejemplo.com/usuarios'](https://api.ejemplo.com/usuarios)
- **Método HTTP:** El tipo de operación (GET, POST, PUT, DELETE)
- **Headers:** Información adicional (tipo de contenido, autenticación)
- **Payload:** Los datos que se envían (en peticiones POST/PUT)
- **Response:** La respuesta del servidor (datos + código de estado)



Ejemplo de Petición API



```
1 GET https://jsonplaceholder.typicode.com/users/1
2 Headers:
3   Content-Type: application/json
4   Authorization: Bearer abc123
```



```
1 {
2   "id": 1,
3   "name": "Leanne Graham",
4   "username": "Bret",
5   "email": "Sincere@april.biz",
6   "phone": "1-770-736-8031 x56442"
7 }
```

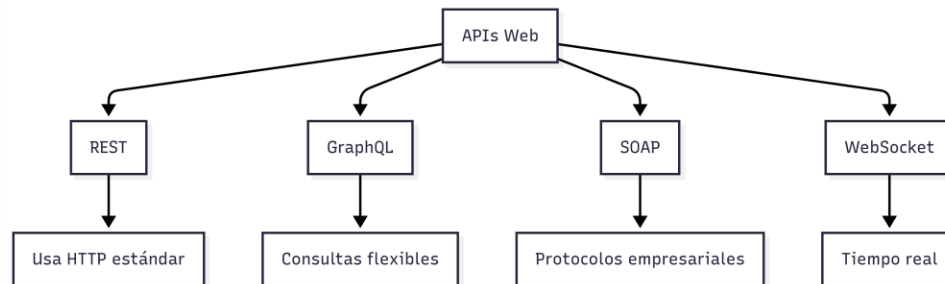


¿Por Qué Son Importantes las APIs?

- **Reutilización:** No reinventar la rueda, usar servicios existentes
- **Especialización:** Cada servicio se enfoca en lo que hace mejor
- **Escalabilidad:** Separar funcionalidades permite crecer de forma modular
- **Mantenimiento:** Actualizar un servicio sin afectar a otros
- **Innovación:** Combinar servicios de diferentes proveedores
- **Tiempo al mercado:** Desarrollar más rápido usando APIs existentes



Principales Tipos de APIs Web

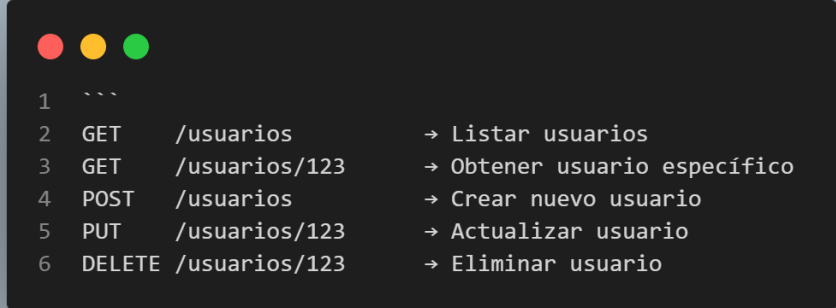


- **REST:** El más popular, usa HTTP de forma estándar
- **GraphQL:** Consultas flexibles, obtén solo lo que necesitas
- **SOAP:** Protocolos empresariales, más rígido pero robusto
- **WebSocket:** Para comunicación en tiempo real



REST – Representational State Transfer

- **Simplicidad:** Usa HTTP tal como fue diseñado
- **Stateless:** Cada petición es independiente
- **Cacheable:** Las respuestas pueden almacenarse en caché
- **Formato:** Principalmente JSON (también XML)
- **Verbos HTTP:** GET; POST; PUT; DELETE corresponden a operaciones CRUD



```
1  ...
2  GET    /usuarios      → Listar usuarios
3  GET    /usuarios/123    → Obtener usuario específico
4  POST   /usuarios      → Crear nuevo usuario
5  PUT    /usuarios/123    → Actualizar usuario
6  DELETE /usuarios/123    → Eliminar usuario
```

GraphQL – Una Alternativa Moderna

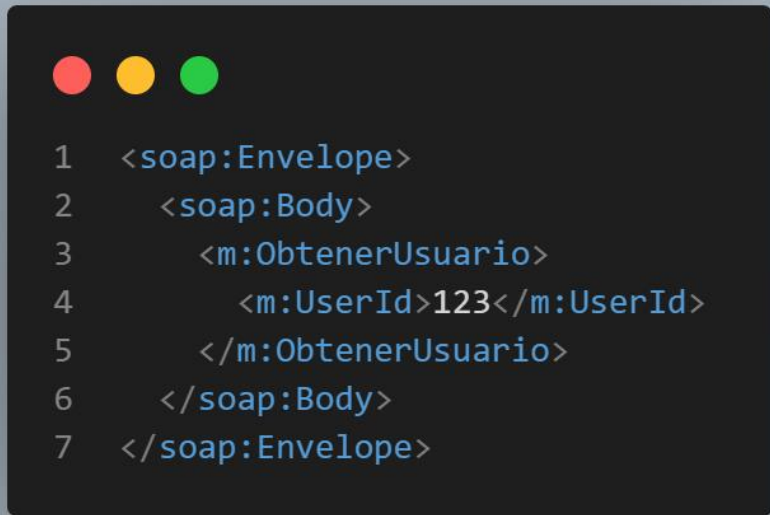


```
1  query {  
2    usuario(id: 123) {  
3      nombre  
4      email  
5      posts {  
6        titulo  
7        fechaCreacion  
8      }  
9    }  
10 }
```

- **Un solo endpoint:** Todas las consultas van a la misma URL
- **Consultas específicas:** Solicita exactamente los datos que necesitas
- **Fuertemente tipado:** Esquema definido claramente
- **Introspección:** La API se auto-documenta

Ventajas: En REST necesitarías múltiples peticiones para obtener estos datos relacionados

SOAP – Simple Object Access Protocol



```
1 <soap:Envelope>
2   <soap:Body>
3     <m:ObtenerUsuario>
4       <m:UserId>123</m:UserId>
5     </m:ObtenerUsuario>
6   </soap:Body>
7 </soap:Envelope>
```

- **Protocolo formal:** Especificación estricta con contratos WSDL (Web Service Description Language)
- **Seguridad robusta:** Estándares WS-Security integrados
- **Transacciones:** Soporte nativo para operaciones complejas
- **Formato XML:** Más verboso, pero muy estructurado
- **Uso típico:** sistemas bancarios, ERP, aplicaciones empresariales

WebSockets - Comunicación Bidireccional

- **Conexión persistente:** No hay que hacer peticiones repetidas
- **Bidireccional:** Cliente y servidor pueden enviar datos en cualquier momento
- **Tiempo real:** Ideal para chat, notificaciones, actualizaciones en vivo
- **Menos overhead:** Después de la conexión inicial, menos datos de protocolo

Casos de uso comunes:

- Chat en tiempo real
- Notificaciones push
- Juegos multijugador online
- Actualizaciones de estado en vivo (bolsa, deportes)
- Colaboración en tiempo real (Google Docs)



¿Cuándo Usar Cada Tipo?

Tipo	Mejor para	Ventajas	Desventajas
REST	CRUD básico, APIs públicas	Simple, amplio soporte	Múltiples peticiones
GraphQL	Apps móviles, datos complejos	Flexibilidad, eficiencia	Curva de aprendizaje
SOAP	Sistemas empresariales	Seguridad, transacciones	Complejo, verboso
WebSocket	Tiempo real	Bidireccional, rápido	Conexión persistente

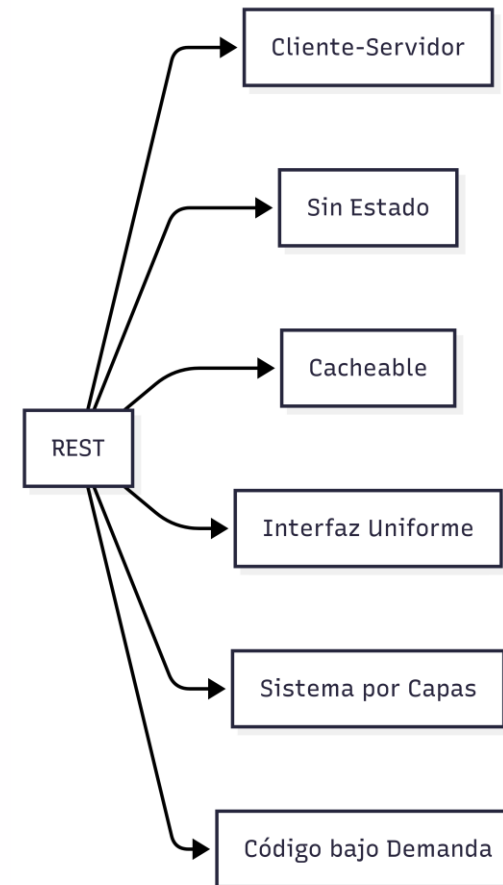


Los 6 Principios de REST



Principios Fundamentales de REST

1. **Cliente-Servidor:** Separación clara de responsabilidades
2. **Sin Estado (Stateless):** Cada petición es independiente
3. **Cacheable:** Las respuestas pueden almacenarse temporalmente
4. **Interfaz Uniforme:** Contratos consistentes
5. **Sistema por Capas:** Arquitectura modular
6. **Código bajo Demanda:** Cliente puede ejecutar código del servidor (opcional)



Separación Cliente-Servidor

Responsabilidades separadas:

- Cliente: Interfaz de usuario, UX
- Servidor: Almacenamiento de datos, lógica de negocio

Beneficios:

- Desarrollo independiente
- Escalabilidad separada
- Portabilidad del cliente
- Evolución independiente

Ejemplo:

Cliente (React/Vue) <--> API REST <--> Servidor (Node.js/Python)

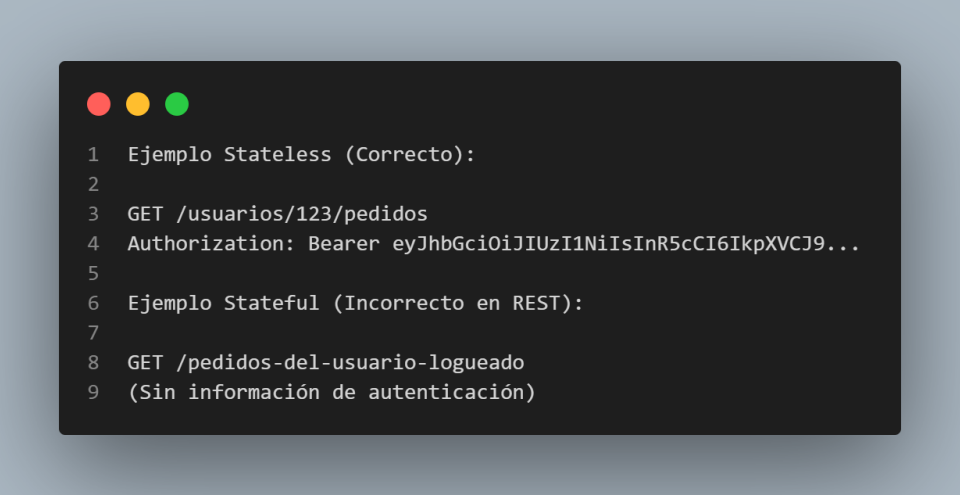


Comunicación Sin Estado

- Cada petición debe contener toda la información necesaria
- El servidor no recuerda peticiones anteriores
- Toda información de contexto va en la petición

Ventajas:

- Escalabilidad horizontal
- Simplicidad del servidor
- Tolerancia a fallos

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays two examples of REST API requests. The first example is labeled 'Ejemplo Stateless (Correcto):' and shows a GET request to '/usuarios/123/pedidos' with a Bearer token in the Authorization header. The second example is labeled 'Ejemplo Stateful (Incorrecto en REST):' and shows a GET request to '/pedidos-del-usuario-logueado' without any authentication information.

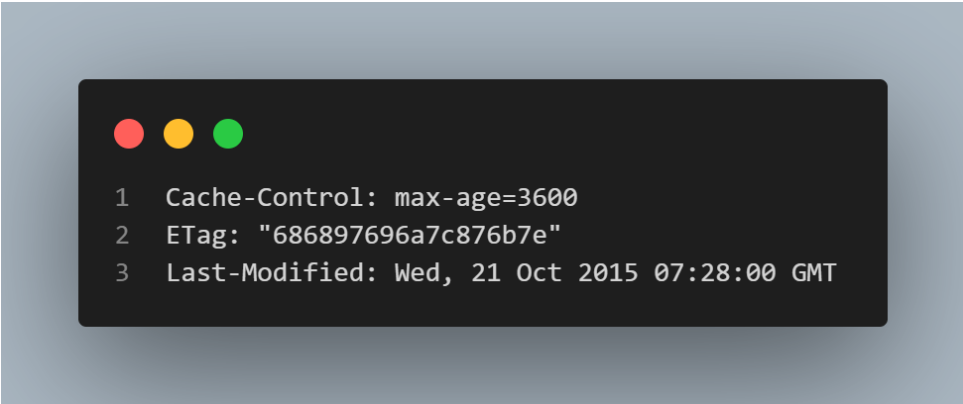
```
1 Ejemplo Stateless (Correcto):  
2  
3 GET /usuarios/123/pedidos  
4 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...  
5  
6 Ejemplo Stateful (Incorrecto en REST):  
7  
8 GET /pedidos-del-usuario-logueado  
9 (Sin información de autenticación)
```

Respuestas Cacheables

- Las respuestas deben indicar si pueden almacenarse en caché
- Mejora el rendimiento y reduce la carga del servidor

Beneficios:

- Menor latencia
- Menos ancho de banda
- Mejor experiencia de usuario
- Reducción de carga del servidor

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays three lines of HTTP headers.

```
1 Cache-Control: max-age=3600
2 ETag: "686897696a7c876b7e"
3 Last-Modified: Wed, 21 Oct 2015 07:28:00 GMT
```

Interfaz Uniforme

Consistencia en toda la API

Cuatro sub-principios:

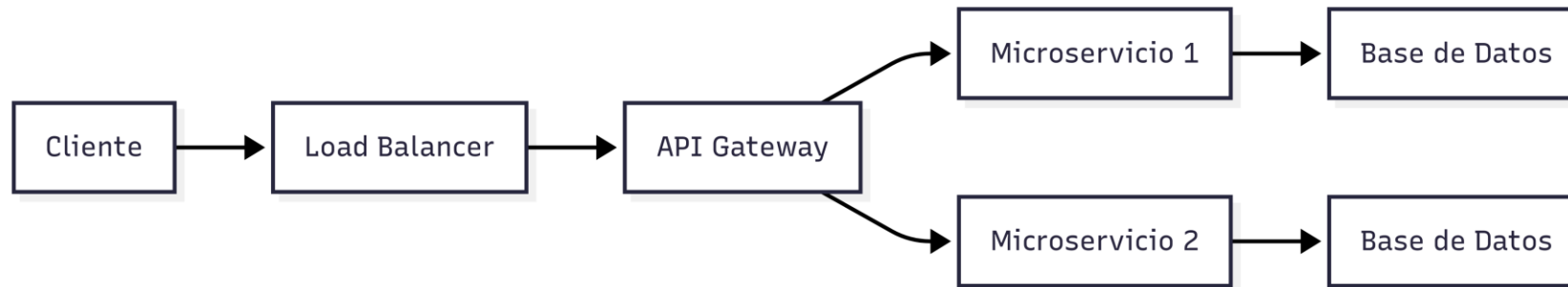
1. **Identificación de recursos:** URLs claras y consistentes
 - '/usuarios/123'
 - '/productos/456'
2. **Manipulación mediante representaciones:** JSON/XML para operaciones
3. **Mensajes auto-descriptivos:** Headers y códigos de estado claros
4. **HATEOAS:** Hipermedia como motor del estado de la aplicación



Arquitectura en Capas

- **Beneficios**

- Escalabilidad
- Seguridad (firewalls, autenticación)
- Balanceador de carga
- Caché intermedio



Buenas Prácticas en Diseño de APIs REST



URLs Bien Diseñadas



Buenas prácticas



Evitar



```
1 GET    /usuarios           → Lista de usuarios
2 GET    /usuarios/123       → Usuario específico
3 GET    /usuarios/123/pedidos → Pedidos del usuario 123
4 POST   /usuarios           → Crear usuario
5 PUT    /usuarios/123      → Actualizar usuario completo
6 PATCH  /usuarios/123    → Actualizar parcialmente
7 DELETE /usuarios/123     → Eliminar usuario
```



```
1 /obtenerUsuarios
2 /crearNuevoUsuario
3 /actualizarUsuario?id=123
4 /borrarUsuario/123
```

Correspondencia HTTP-CRUD

Verbo HTTP	Operación CRUD	Idempotente	Seguro
GET	Read (Leer)	✓	✓
POST	Create (Crear)	✗	✗
PUT	Update (Actualizar completo)	✓	✗
PATCH	Update (Actualizar parcial)	✗	✗
DELETE	Delete (Eliminar)	✓	✗

Conceptos clave:

- **Idempotente:** Múltiples llamadas = Mismo resultado
- **Seguro:** No modifica el estado del servidor

Principales Códigos de Respuesta

- **2xx - Éxito**
 - 200 OK - Petición exitosa
 - 201 Created – Recurso creado
 - 204 No content - Éxito sin contenido
- **4xx – Error del cliente**
 - 400 Bad Request - Petición mal formada
 - 401 Unauthorized – No autenticado
 - 403 Forbidden – sin permisos
 - 404 Not Found – Recurso no encontrado
- **5xx – Error del servidor**
 - 500 Internal Server Error – Error interno
 - 503 Service Unavailable – Servicio no disponible



Formato Consistente de Respuestas

Respuesta exitosa

```
1 {  
2   "success": true,  
3   "data": {  
4     "id": 123,  
5     "nombre": "Juan Pérez",  
6     "email": "juan@ejemplo.com"  
7   },  
8   "message": "Usuario obtenido correctamente"  
9 }
```

Respuesta de error

```
1 {  
2   "success": false,  
3   "error": {  
4     "code": "VALIDATION_ERROR",  
5     "message": "Email inválido",  
6     "details": {  
7       "field": "email",  
8       "value": "correo-invalido"  
9     }  
10  }  
11 }
```

Manejo de Grandes Conjuntos de Datos

- **Paginación:**
 - GET /usuarios?page=2&limit=10
- **Filtrado y ordenación:**
 - GET /productos?categoria=electronic
a&precio_max=500&sort=precio,
-fecha
- **Respuesta con metadatos ---->**

```
1  {
2    "data": [...],
3    "pagination": {
4      "currentPage": 2,
5      "totalPages": 15,
6      "totalItems": 150,
7      "itemsPerPage": 10
8    }
9  }
```

Evolución de APIs sin Romper Clientes

Estrategias de versionado

- **URL Path:**
 - /v1/usuarios
 - /v2/usuarios
- **Parameter**
 - /usuarios?version=2
- **Header**
 - Accept: application/vnd.api+json;version=2

Mejores prácticas

- Mantener retrocompatibilidad cuando sea posible
- Documentar cambios claramente
- Dar tiempo de migración a los clientes



Seguridad en APIs REST

Métodos de autenticación:

1. API Keys:

- Authorization: Bearer sk_live_123456789

2. JWT (JSON Web Tokens):

- Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

3. OAuth 2.0:

- Para acceso en nombre de usuarios
- Permisos granulares (scopes)

Buenas prácticas de seguridad

1. HTTPS obligatorio
2. Rate limiting
3. Validación de entrada
4. Logs de seguridad



Documentación Clara y Completa

- **Elementos esenciales**

- Descripción de cada endpoint
- Parámetros requeridos y opcionales
- Ejemplos de peticiones y respuestas
- Códigos de error posibles
- Guías de inicio rápido

- **Herramientas populares:**

- Swagger/OpenAPI: Estándar de la industria
- Postman: Colecciones interactivas
- GitBook: Documentación narrativa

```
1  paths:
2    /usuarios:
3      get:
4        summary: Obtener lista de usuarios
5        parameters:
6          - name: page
7            in: query
8            type: integer
9        responses:
10         200:
11           description: Lista de usuarios
```

Asegurar Calidad en APIs

Tipos de testing

1. **Testing unitario:** Lógica de cada endpoint
2. **Testing de integración:** Flujos completos
3. **Testing de rendimiento:** Carga y estrés
4. **Testing de seguridad:** Vulnerabilidades

Herramientas

- Postman: Testing manual y automatizado
- Jest/Mocha: Testing unitario
- Artillery: Testing de carga
- OAWSP ZAP: Testing de seguridad

```
1 test('GET /usuarios debe retornar lista de usuarios', async () => {  
2   const response = await request(app).get('/usuarios');  
3   expect(response.status).toBe(200);  
4   expect(response.body.data).toBeInstanceOf(Array);  
5 });
```

El Futuro de las APIs

Tendencias emergentes:

- **APIs de AI/ML:** Integración de inteligencia artificial (por ejemplo, con protocolo MCP)
- **Real-time APIs:** WebSockets y Server-Sent Events
- **Más GraphQL:** Crecimiento en aplicaciones complejas
- **Desarrollo API-First:** Diseñar la API antes que la UI
- **Low-code/No-code:** Herramientas visuales para APIs (por ejemplo, servicios como API Gateway en AWS)



kingscorner